EXPERIMENT-02

AIM: To implement linear search in Python and analyze its best, average, and worst cases using medical records from the Breast Cancer Diagnosis dataset..

Theory:

Linear search is used on a collections of items. It relies on the technique of traversing a list from start to end by exploring properties of all the elements that are found on the way.

For example, consider an array of integers of size . You should find and print the position of all the elements with value . Here, the linear search is based on the idea of matching each element from the beginning of the list to the end of the list with the integer , and then printing the position of the element if the condition is `True'.

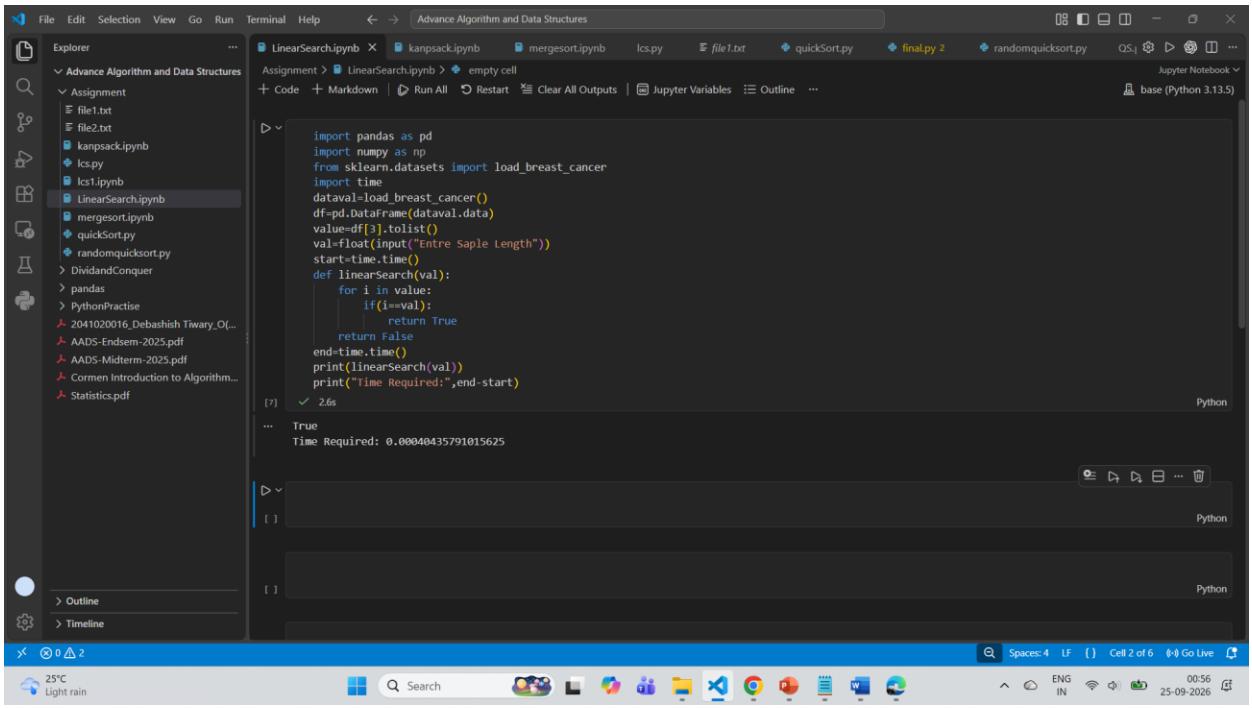
.

Code:

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
import time
dataval=load_breast_cancer()
df=pd.DataFrame(dataval.data)
value=df[3].tolist()
val=float(input("Entre Saple Length"))
start=time.time()
def linearSearch(val):
 for i in value:
    if(i==val):
        return True
 return False
end=time.time()
print(linearSearch(val))
print("Time Required:",end-start)
```

Complexity Analysis:
Best Case = O(1) – If data is present at very first index
Worst Case= O(n) – If data is present at last index or not present at all.

OUTPUT:


File Edit Selection View Go Run Terminal Help
Advance Algorithm and Data Structures
Explorer
Advance Algorithm and Data Structures
Assignment
file1.txt
file2.txt
kanpsack.ipynb
lcs.py
lcs1.ipynb
LinearSearch.ipynb
mergesort.ipynb
quickSort.py
randomquicksort.py
DividandConquer
pandas
PythonPractise
2041020016_Debashish Tiwary_O(...
AADS-Endsem-2025.pdf
AADS-Midterm-2025.pdf
Cormen Introduction to Algorithm...
Statistics.pdf
Outline
Timeline
LinearSearch.ipynb
kanpsack.ipynb
mergesort.ipynb
lcs.py
file1.txt
quickSort.py
final.py 2
randomquicksort.py
Assignment > LinearSearch.ipynb > empty cell
Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Jupyter Notebook
base (Python 3.13.5)
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
import time
dataval=load_breast_cancer()
df=pd.DataFrame(dataval.data)
value=df[3].tolist()
val=float(input("Entre Saple Length"))
start=time.time()
def linearSearch(val):
for i in value:
if(i==val):
return True
return False
end=time.time()
print(linearSearch(val))
print("Time Required:",end-start)
[7] 2.6s
Python
True
Time Required: 0.0004043579101562 5
[ ]
Python
[ ]
Python
0 2
Spaces: 4 LF { } Cell 2 of 6 Go Live
25°C
Light rain
Search
ENG
IN
00:56
25-09-2026

EXPERIMENT-02

AIM: To implement the Fourier Transform using the divide-and-conquer technique in Python.

Theory:

The Fourier Transform converts a signal from the time domain into the frequency domain or vice versa. For example, a signal may contain several frequencies mixed together. In the time domain, these frequencies are difficult to identify. After applying the Fourier Transform, the contribution of each frequency becomes visible. In other words, instead of asking how the signal changes moment by moment, it disentangles which pure tones or repeating patterns are mixed together to make the signal. Think of it like a prism separating white light into its component colors.

For a sequence containing Nsamples, the Discrete Fourier Transform is

$N-1$ $X[k] = \sum xn=0$

where:

- $x[n]$is the input signal.
- $X[k]$is the frequency-domain output.
- $N$is the number of samples.
- $j=\sqrt{-1}$. $[n]e-j2\pi kn/N$

Directly implementing this equation requires approximately N^2 calculations.

Its time complexity is therefore $O(N$^2).

The Fast Fourier Transform (FFT) calculates the same result more efficiently using divide and conquer.

Uses

- Audio and Image Processing: Compresses MP3 files and JPEGs.
- Telecommunications: Decodes Wi-Fi, radio, and digital cable signals.
- Science and Medicine: Powers MRI scans, seismic monitoring, and vibration analysis.

Divide-and-conquer idea: The input sequence is divided into two smaller sequences:

- Samples at even indices
- Samples at odd indices

For example: Input: [1, 2, 3, 4] Even-indexed samples: [1, 3] Odd-indexed samples: [2, 4] The Fourier Transform is recursively calculated for both parts. Their results are then combined using complex exponential factors called twiddle factors. The process is: 1. Divide the input into even- and odd-indexed samples. 2. Recursively calculate the FFT of both parts. 3. Combine their results. 4. Stop when only one element remains. .

.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


#Loading real weather data
file_path = r"Assignment/india_2000_2024_daily_weather.csv"


df = pd.read_csv(file_path)
df["date"] = pd.to_datetime(df["date"])


#Using delhi
city = "Kolkata"


data = (df[df["city"] == city].sort_values("date").reset_index(drop=True))


print("City:", city)
print("Observations:", len(data))
print("Date range:", data["date"].min().date(),"to", data["date"].max().date())


#1)Fourier Series : Temperature Reconstruction
temperature = data["temperature_2m_max"].to_numpy(dtype=float)


N = len(temperature)
t = np.arange(N)


# Approximate number of days in one year
```

```
YEAR = 365.2425


# Number of Fourier harmonics
K = 6


# Constant component
a0 = np.mean(temperature)
reconstructed = np.full(N, a0)
for n in range(1, K + 1):
 an = (2 / N) * np.sum(temperature *np.cos(2*np.pi*n*t/YEAR))  bn
= (2 / N)*np.sum(temperature*np.sin(2*np.pi*n*t/ YEAR))


    reconstructed += (an * np.cos(2 * np.pi * n * t / YEAR)+ bn * np.sin(2 * np.pi * n * t / YEAR))


# Calculate errors
rmse = np.sqrt(np.mean((temperature - reconstructed) ** 2))
mae = np.mean(np.abs(temperature - reconstructed))


print("\nFOURIER SERIES")
print("Variable: Daily Maximum Temperature")
print("Harmonics:", K)
print("RMSE:", round(rmse, 3), "°C")
print("MAE :", round(mae, 3), "°C")


# Plot actual vs reconstructed temperature
plt.figure(figsize=(14, 5))


plt.plot(data["date"],temperature,label="Actual Temperature",linewidth=0.7)
```

```
plt.plot(data["date"],reconstructed,label="Fourier Series Reconstruction",linewidth=2)


plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.title("Delhi Temperature - Fourier Series Reconstruction")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()


#2)DFT/FFT : Rainfall Frequency Analysis
rainfall = data["precipitation_sum"].to_numpy(dtype=float)


# Remove mean
rainfall_centered = rainfall - np.mean(rainfall)


# FFT computes the DFT efficiently
fft_result = np.fft.fft(rainfall_centered)


# Sampling interval = 1 day
frequencies = np.fft.fftfreq(N, d=1)


# Only positive frequencies
positive = frequencies > 0


freq = frequencies[positive]
amplitude = (2 / N) * np.abs(fft_result[positive])
```

```
# Strongest frequency
index = np.argmax(amplitude)


dominant_frequency = freq[index]
dominant_period = 1 / dominant_frequency


print("\nDFT / FFT")
print("Variable: Daily Rainfall")
print("Dominant Frequency:",round(dominant_frequency, 8),"cycles/day")


print("Dominant Period:",round(dominant_period, 2),"days")


print("Dominant Period:",round(dominant_period / 30.44, 2),"months")


# Frequency spectrum
plt.figure(figsize=(14, 5))


plt.plot(freq, amplitude)


plt.xlabel("Frequency (cycles/day)")
plt.ylabel("Amplitude")
plt.title("Delhi Rainfall - DFT / FFT Frequency Spectrum")


# Show low-frequency region clearly
plt.xlim(0, 0.1)


plt.grid(True)
```

```
plt.tight_layout()
plt.show()


#3)Doing extra fft on temp
temperature_centered = temperature - np.mean(temperature)
temperature_fft = np.fft.fft(temperature_centered)
temperature_freq = np.fft.fftfreq(N, d=1)
positive_temp = temperature_freq > 0
temp_freq = temperature_freq[positive_temp]
temp_amplitude = (2 / N *np.abs(temperature_fft[positive_temp]))
temp_index = np.argmax(temp_amplitude)
temp_dominant_frequency = temp_freq[temp_index]
temp_dominant_period = 1 / temp_dominant_frequency


print("\nEXTRA FFT")
print("Variable: Temperature")
print("Dominant Frequency:",round(temp_dominant_frequency, 8),"cycles/day")


print("Dominant Period:",round(temp_dominant_period, 2),"days")


# Temperature frequency spectrum
plt.figure(figsize=(14, 5))


plt.plot(temp_freq, temp_amplitude)


plt.xlabel("Frequency (cycles/day)")
plt.ylabel("Amplitude")
plt.title("Delhi Temperature - DFT / FFT Frequency Spectrum")
```

```
plt.xlim(0, 0.02)


plt.grid(True)

plt.tight_layout()

plt.show()


#Final Result

print("\nFINAL RESULT")

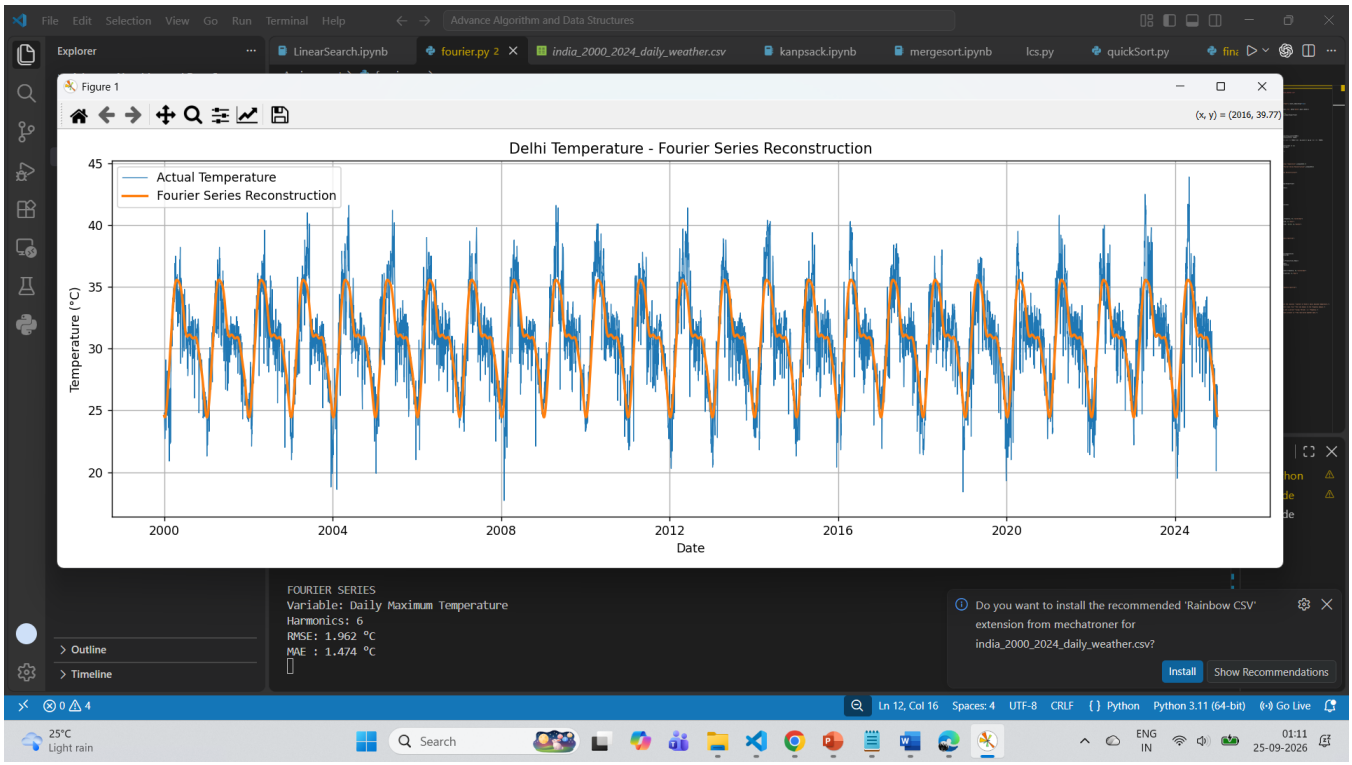

print("Fourier Series was used to reconstruct the seasonal ""pattern of Delhi's daily maximum temperature.")

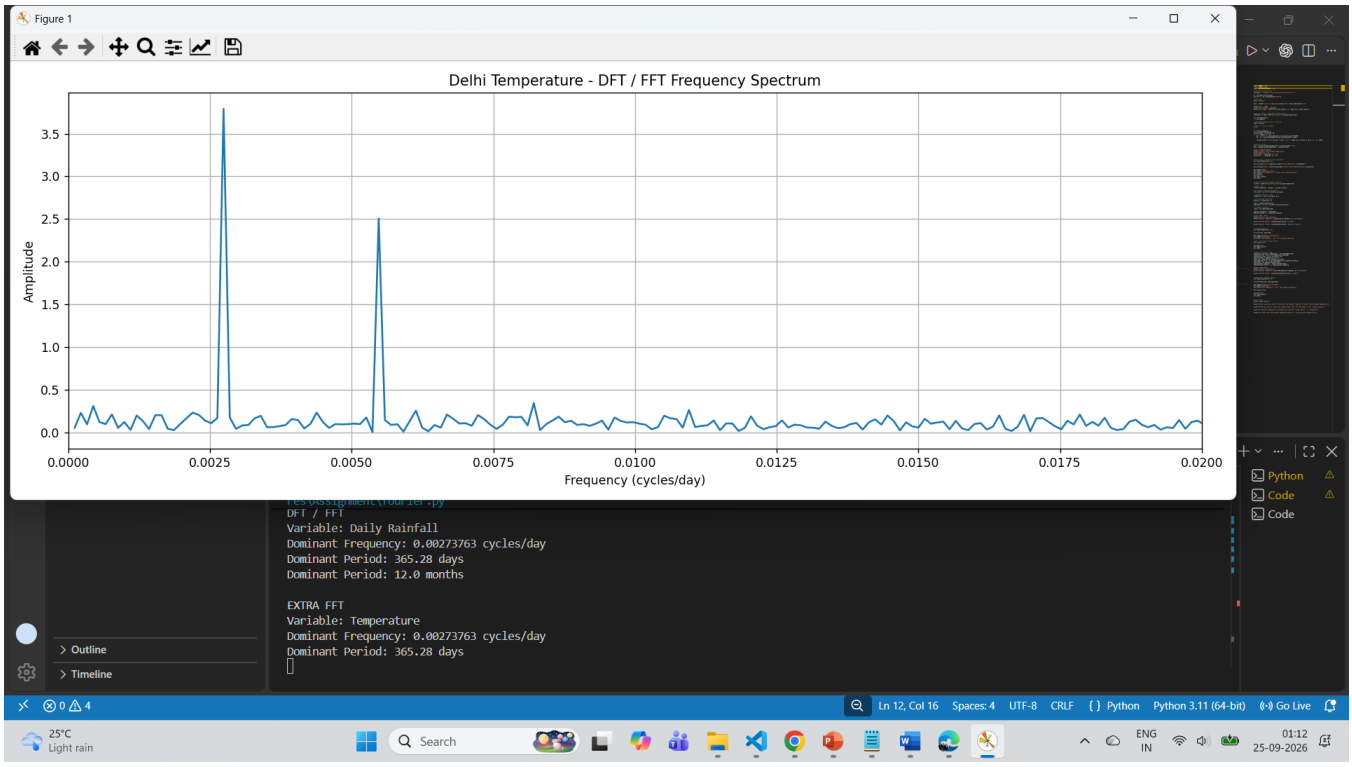

print("DFT/FFT was used to convert the rainfall data from ""the time domain to the frequency domain.")


print("The dominant frequency was converted into a period ""using: Period = 1 / Frequency.")


print("The results show the periodic components present in ""the real-world weather data.")
```

OUTPUT:


Figure 1
Delhi Temperature - Fourier Series Reconstruction
Actual Temperature
Fourier Series Reconstruction
Temperature (°C)
Date
FOURIER SERIES
Variable: Daily Maximum Temperature
Harmonics: 6
RMSE: 1.962 °C
MAE : 1.474 °C
Do you want to install the recommended 'Rainbow CSV' extension from mechatroner for india_2000_2024_daily_weather.csv?
Install
Show Recommendations


Figure 1
Delhi Temperature - DFT / FFT Frequency Spectrum
Amplitude
Frequency (cycles/day)
DFT / FFT
Variable: Daily Rainfall
Dominant Frequency: 0.00273763 cycles/day
Dominant Period: 365.28 days
Dominant Period: 12.0 months
EXTRA FFT
Variable: Temperature
Dominant Frequency: 0.00273763 cycles/day
Dominant Period: 365.28 days

EXPERIMENT-03

AIM: To implement the Merge Sort using the divide-and-conquer technique in Python.

Theory:

Merge sort is a divide-and-conquer algorithm based on the idea of breaking down a list into several sub-lists until each sublist consists of a single element and merging those sublists in a manner that results into a sorted list.

Idea:

- Divide the unsorted list into sublists, each containing element.
- Take adjacent pairs of two singleton lists and merge them to form a list of 2 elements. will now convert into lists of size 2.
- Repeat the process till a single sorted list of obtained.

While comparing two sublists for merging, the first element of both lists is taken into consideration. While sorting in ascending order, the element that is of a lesser value becomes a new element of the sorted list. This procedure is repeated until both the smaller sublists are empty and the new combined sublist comprises all the elements of both the sublists.

Code:

```
import random
data=[]
for i in range(1,100):
 data.append(random.randint(-1000,1000))
print("Unsorted:",data)
def divide(data,left,right):
 if(left<right):
    mid=(left+right)//2
    divide(data,left,mid)
    divide(data,mid+1,right)
    conquer(data,left,mid,right)
def conquer(data,left,mid,right):
 leftpart=data[left:mid+1]
 rightpart=data[mid+1:right+1]
 i=0
```

```
    j=0
    k=left
    while i < len(leftpart) and j < len(rightpart):
        if(leftpart[i]<=rightpart[j]):
            data[k]=leftpart[i]
            i+=1
        else:
            data[k]=rightpart[j]
            j+=1
        k+=1
    while i < len(leftpart):
        data[k] = leftpart[i]
        i += 1
        k += 1

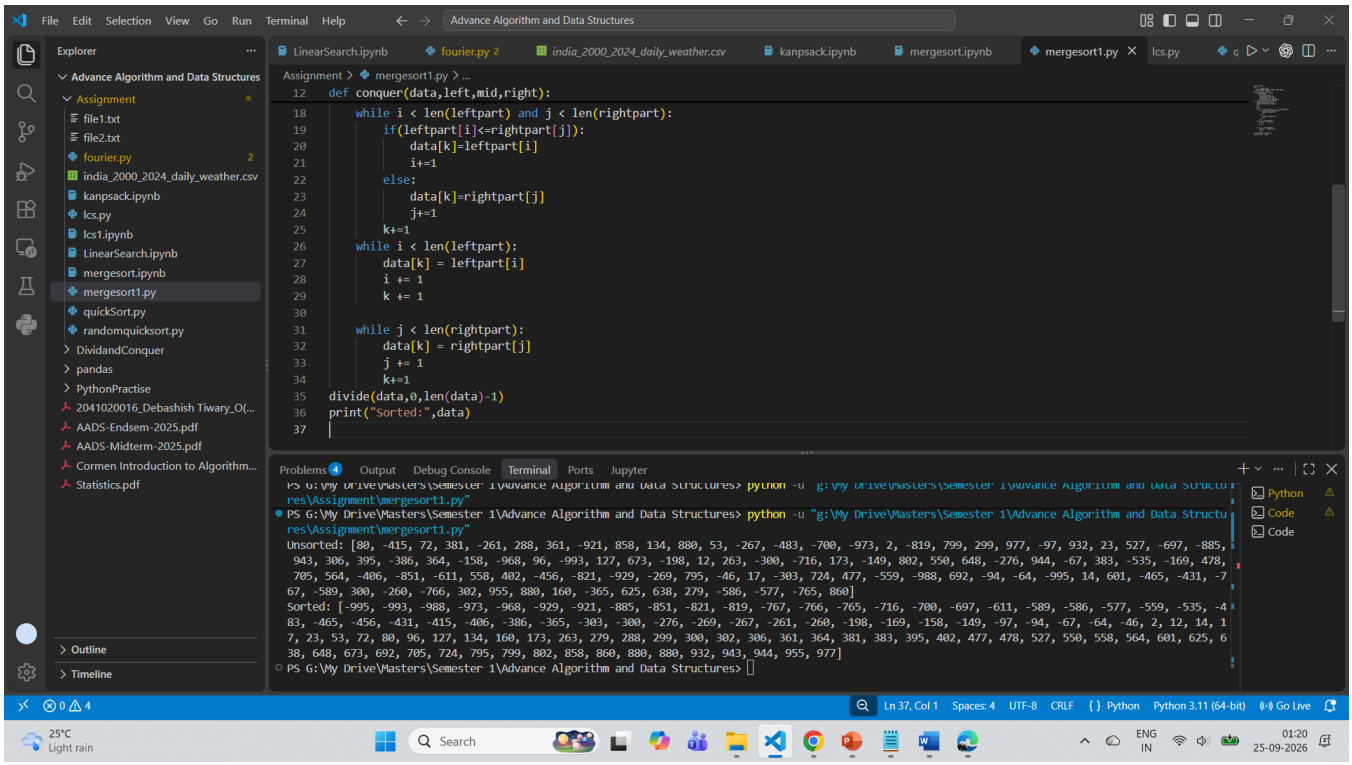

 while j < len(rightpart):
    data[k] = rightpart[j]
    j += 1
    k+=1
divide(data,0,len(data)-1)
print("Sorted:",data)
```

Complexity Analysis:

Time Complexity: Merge Sort takes O(NlogN) time in all cases considering representation of data in any order

Space Complexity: It takes an extra space of O(N) (N: length of data) and also auxiliary space due to recursion call.

OUTPUT:

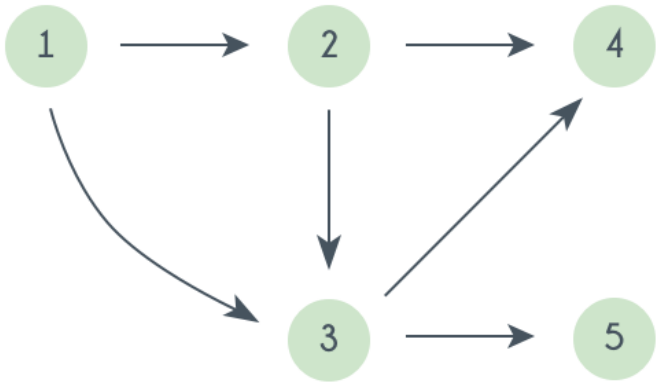

File Edit Selection View Go Run Terminal Help
Advance Algorithm and Data Structures
Explorer
Advance Algorithm and Data Structures
Assignment
file1.txt
file2.txt
fourier.py
india_2000_2024_daily_weather.csv
kanpsack.ipynb
lcs.py
lcs1.ipynb
LinearSearch.ipynb
mergesort.ipynb
mergesort1.py
quickSort.py
randomquicksort.py
DividandConquer
pandas
PythonPractise
2041020016_Debashish Tiwary_O(...
AADS-Endsem-2025.pdf
AADS-Midterm-2025.pdf
Cormen Introduction to Algorithm...
Statistics.pdf
Outline
Timeline
LinearSearch.ipynb
fourier.py 2
india_2000_2024_daily_weather.csv
kanpsack.ipynb
mergesort.ipynb
mergesort1.py
lcs.py
Assignment > mergesort1.py > ...
def conquer(data,left,mid,right):
while i < len(leftpart) and j < len(rightpart):
if(leftpart[i]<=rightpart[j]):
data[k]=leftpart[i]
i+=1
else:
data[k]=rightpart[j]
j+=1
k+=1
while i < len(leftpart):
data[k] = leftpart[i]
i += 1
k += 1
while j < len(rightpart):
data[k] = rightpart[j]
j += 1
k+=1
divide(data,0,len(data)-1)
print("Sorted:",data)
Problems 4
Output
Debug Console
Terminal
Ports
Jupyter
PS G:\My Drive\Masters\Semester 1\Advance Algorithm and Data Structures> python -u "g:\My Drive\Masters\Semester 1\Advance Algorithm and Data Structures\Assignment\mergesort1.py"
Unsorted: [80, -415, 72, 381, -261, 288, 361, -921, 858, 134, 880, 53, -267, -483, -700, -973, 2, -819, 799, 299, 977, -97, 932, 23, 527, -697, -885, 943, 306, 395, -386, 364, -158, -968, 96, -993, 127, 673, -198, 12, 263, -300, -716, 173, -149, 802, 550, 648, -276, 944, -67, 383, -535, -169, 478, 705, 564, -406, -851, -611, 558, 402, -456, -821, -929, -269, 795, -46, 17, -303, 724, 477, -559, -988, 692, -94, -64, -995, 14, 601, -465, -431, -767, -589, 300, -260, -766, 302, 955, 880, 160, -365, 625, 638, 279, -586, -577, -765, 860]
Sorted: [-995, -993, -988, -973, -968, -929, -921, -885, -851, -821, -819, -767, -766, -765, -716, -700, -697, -611, -589, -586, -577, -559, -535, -483, -465, -456, -431, -415, -406, -386, -365, -303, -300, -276, -269, -267, -261, -260, -198, -169, -158, -149, -97, -94, -67, -64, -46, 2, 12, 14, 17, 23, 53, 72, 80, 96, 127, 134, 160, 173, 263, 279, 288, 299, 300, 302, 306, 361, 364, 381, 383, 395, 402, 477, 478, 527, 550, 558, 564, 601, 625, 638, 648, 673, 692, 705, 724, 795, 799, 802, 858, 860, 880, 880, 932, 943, 944, 955, 977]
PS G:\My Drive\Masters\Semester 1\Advance Algorithm and Data Structures>
Python
Code
Code
Ln 37, Col 1
Spaces: 4
UTF-8
CRLF
Python
Python 3.11 (64-bit)
Go Live
25°C
Light rain
Search
ENG
IN
01:20
25-09-2026

EXPERIMENT-04

AIM: To implement the Topological Sorting is Directed Acyclic Graph in Python.

Theory:

Topological sorting of vertices of a **Directed Acyclic Graph** is an ordering of the vertices $v_1, v_2, \dots v_n$ in such a way, that if there is an edge directed towards vertex v_j from vertex v_i, then v_i comes before v_j. For example consider the graph given below:



A topological sorting of this graph is: 1 2 3 4 5
There are multiple topological sorting possible for a graph. For the graph given above one another topological sorting is: 1 2 3 5 4
In order to have a topological sorting the graph must not contain any cycles. In order to prove it, let's assume there is a cycle made of the vertices $v_1, v_2, v_3 \dots v_n$. That means there is a directed edge between v_i and v_{i+1} $(1 \leq i < n)$ and between v_n and v_1. So now, if we do topological sorting then v_n must come before v_1 because of the directed edge from v_n to v_1. Clearly, v_{i+1} will come after v_i, because of the directed from v_i to v_{i+1}, that means v_1 must come before v_n. Well, clearly we've reached a contradiction, here. So topological sorting can be achieved for only directed and acyclic graphs.

Le'ts see how we can find a topological sorting in a graph. So basically we want to find a permutation of the vertices in which for every vertex v_i, all the vertices v_j having edges coming out and directed towards v_i comes before v_i. We'll maintain an array T that will denote our topological sorting. So, let's say for a graph having N vertices, we have an array $in_degree[]$ of size N whose i^{th} element tells the number of vertices which are not already inserted in T and there is an edge from them incident on vertex numbered i. We'll append vertices v_i to the array T, and when we do that we'll decrease the value of $in_degree[v_j]$ by 1 for every edge from v_i to v_j. Doing this will mean that we have inserted one vertex having edge directed towards v_j. So at any point we can insert only those vertices for which the value of $in_degree[]$ is 0.

Code:

```
from collections import defaultdict, deque


class Graph:
 def __init__(self, vertices):
    self.V = vertices
    self.adj = defaultdict(list)
 def add_edge(self, u, v):
    """Adds a directed edge from vertex u to vertex v (u -> v)."""
self.adj[u].append(v)
```

```
 def topological_sort(self):
    """Performs Topological Sort using Kahn's Algorithm (Indegree/BFS
approach)."""          # Step 1: Calculate in-degrees of all vertices
    in_degree = [0] * self.V
    for node in range(self.V):
        for neighbor in self.adj[node]:
            in_degree[neighbor] += 1


    # Step 2: Queue all vertices with in-degree 0
    queue = deque([i for i in range(self.V) if in_degree[i] ==
   0])


    topo_order = []


    # Step 3: Process nodes in queue
    while queue:
        node = queue.popleft()
        topo_order.append(node)


        # Reduce in-degree of neighboring nodes
        for neighbor in self.adj[node]:
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            queue.append(neighbor)


    # Step 4: Check for cycle dependency
    if len(topo_order) != self.V:
        print("Error: The graph contains a cycle! Topological sort is not possible.")
        return []


    return topo_order
```

```
if __name__ == "__main__":

    g = Graph(6)



    g.add_edge(5, 2)
    g.add_edge(5, 0)
    g.add_edge(4, 0)
    g.add_edge(4, 1)
    g.add_edge(2, 3)
    g.add_edge(3, 1)


    print("Graph adjacency list:")
    for node in range(g.V):
        print(f"{node} -> {g.adj[node]}")


    print("\nPerforming Topological Sort...")
    result = g.topological_sort()

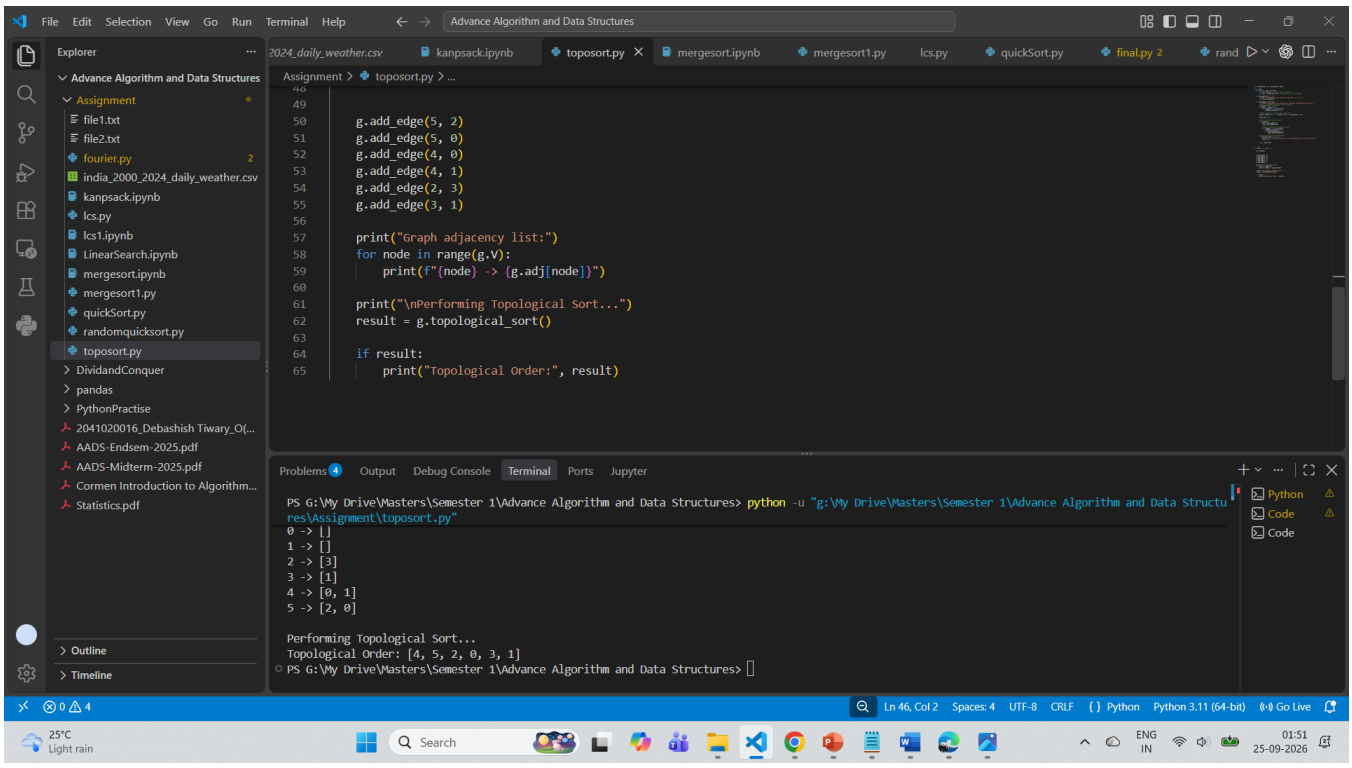

    if result:
        print("Topological Order:", result)
```

Complexity Analysis:

Time Complexity: It generally takes O(NlogN) time mainly dominated by the sorting approach that we take.

Space Complexity: It takes an extra space of O(1) or O(N) based on situation or desired outcomes.

OUTPUT:


File Edit Selection View Go Run Terminal Help
Advance Algorithm and Data Structures
Explorer
Advance Algorithm and Data Structures
Assignment
file1.txt
file2.txt
fourier.py
india_2000_2024_daily_weather.csv
kanpsack.ipynb
lcs.py
lcs1.ipynb
LinearSearch.ipynb
mergesort.ipynb
mergesort1.py
quickSort.py
randomquicksort.py
toposort.py
DividandConquer
pandas
PythonPractise
2041020016_Debashish Tiwary_O(...
AADS-Endsem-2025.pdf
AADS-Midterm-2025.pdf
Cormen Introduction to Algorithm...
Statistics.pdf
Outline
Timeline
2024_daily_weather.csv
kanpsack.ipynb
toposort.py
mergesort.ipynb
mergesort1.py
lcs.py
quickSort.py
final.py 2
rand
Assignment > toposort.py > ...
g.add_edge(5, 2)
g.add_edge(5, 0)
g.add_edge(4, 0)
g.add_edge(4, 1)
g.add_edge(2, 3)
g.add_edge(3, 1)
print("Graph adjacency list:")
for node in range(g.V):
print(f"{node} -> {g.adj[node]}")
print("\nPerforming Topological Sort...")
result = g.topological_sort()
if result:
print("Topological Order:", result)
Problems 4
Output
Debug Console
Terminal
Ports
Jupyter
PS G:\My Drive\Masters\Semester 1\Advance Algorithm and Data Structures> python -u "g:\My Drive\Masters\Semester 1\Advance Algorithm and Data Structures\Assignment\toposort.py"
0 -> []
1 -> []
2 -> [3]
3 -> [1]
4 -> [0, 1]
5 -> [2, 0]
Performing Topological Sort...
Topological Order: [4, 5, 2, 0, 3, 1]
PS G:\My Drive\Masters\Semester 1\Advance Algorithm and Data Structures>
Python
Code
Code
0 4
Ln 46, Col 2
Spaces: 4
UTF-8
CRLF
Python
Python 3.11 (64-bit)
Go Live
25°C
Light rain
Search
ENG
IN
01:51
25-09-2026

EXPERIMENT-03

AIM: To implement the Quick Sort using the divide-and-conquer technique in Python.

Theory:Quick sort is based on the divide-and-conquer approach based on the idea of choosing one element as a pivot element and partitioning the array around it such that: Left side of pivot contains all the elements that are less than the pivot element Right side contains all elements greater than the pivot

It reduces the space complexity and removes the use of the auxiliary array that is used in merge sort. Selecting a random pivot in an array results in an improved time complexity in most of the cases.

Below is the approach:

Divide: Partition (rearrange) the array $A[p..r]$ into two (possibly empty) subarrays $A[p..q-1]$ and $A[q+1..r]$ such that each element of $A[p..q-1]$ is less than or equal to $A[q]$, which is, in turn, less than or equal to each element of $A[q+1..r]$. Compute the index q as part of this partitioning procedure.

Conquer: Sort the two subarrays $A[p..q-1]$ and $A[q+1..r]$ by recursive calls to quicksort.

Combine: Because the subarrays are already sorted, no work is needed to combine them: the entire array $A[p..r]$ is now sorted.

Code:

```
import random
def partition(low,high,nums):
 pivot=low
 while(low<=high):
   while(low<=high and nums[low]<=nums[pivot]):
        low+=1
   while(low<=high and nums[high]>nums[pivot]):
        high-=1
   if(low<high):
        swap(low,high,nums)
 swap(pivot,high,nums)
 return high
```

```
 temp=nums[low]
 nums[low]=nums[high]
 nums[high]=temp
def quicksort(low,high,nums):
 if(low<high):
    partition_point=partition(low,high,nums)
    quicksort(low,partition_point-1,nums)
    quicksort(partition_point+1,high,nums)
def main():
 list=[]
 for i in range(0,100):
    list.append(random.randint(-100,100))
 print("Unsorted:",list)
 quicksort(0,len(list)-1,list)
 print("Sorted:",list)
if __name__ == "__main__":
 main()
```

Randomized Quick Sort:

```
import random
def partition(low,high,nums):
 pivot=random.randint(low,high)
 while(low<=high):
    while(low<=high and nums[low]<=nums[pivot]):
            low+=1
    while(low<=high and nums[high]>nums[pivot]):
            high-=1
    if(low<high):
            swap(low,high,nums)
 swap(pivot,high,nums)
```

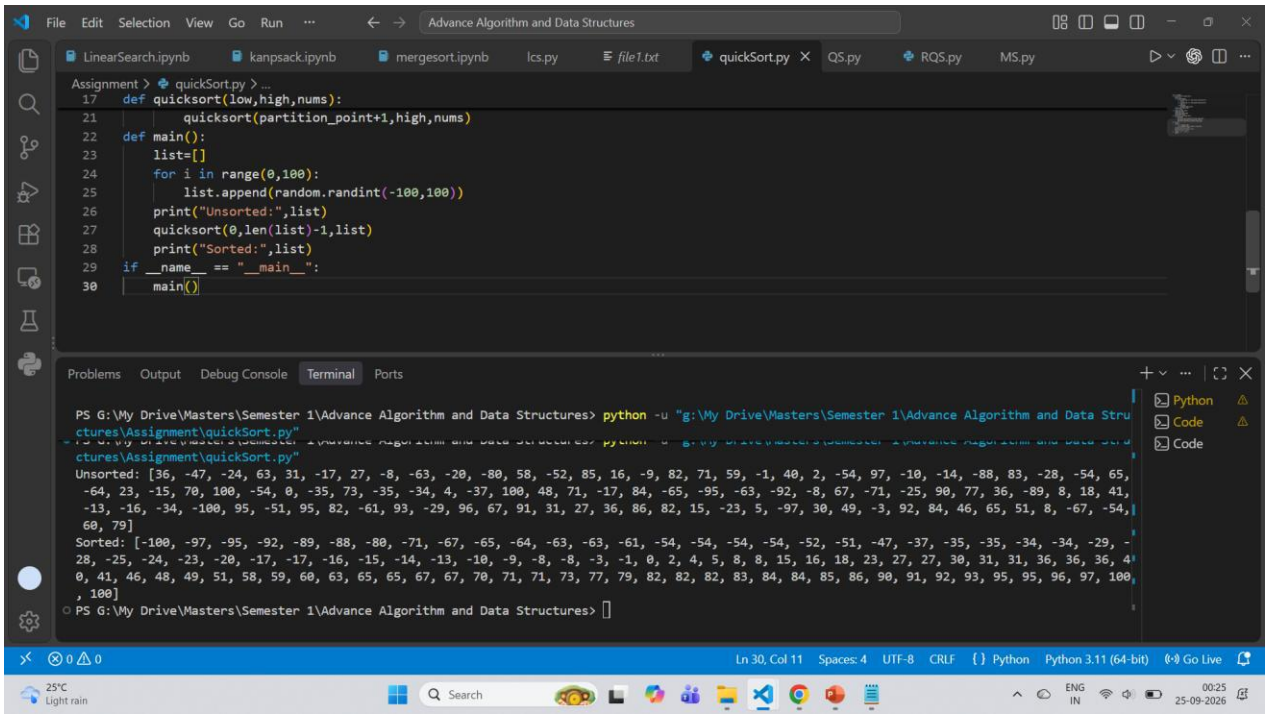
```
 return high
def swap(low,high,nums):
 temp=nums[low]
 nums[low]=nums[high]
 nums[high]=temp
def quicksort(low,high,nums):
 if(low<high):
    partition_point=partition(low,high,nums)
    quicksort(low,partition_point-1,nums)
    quicksort(partition_point+1,high,nums)
def main():
 list=[]
 for i in range(0,100):
    list.append(random.randint(-100,100))
 print("Unsorted:",list)
 quicksort(0,len(list)-1,list)
 print("Sorted:",list)
if __name__ == "__main__":
 main()
```

Complexity Analysis:

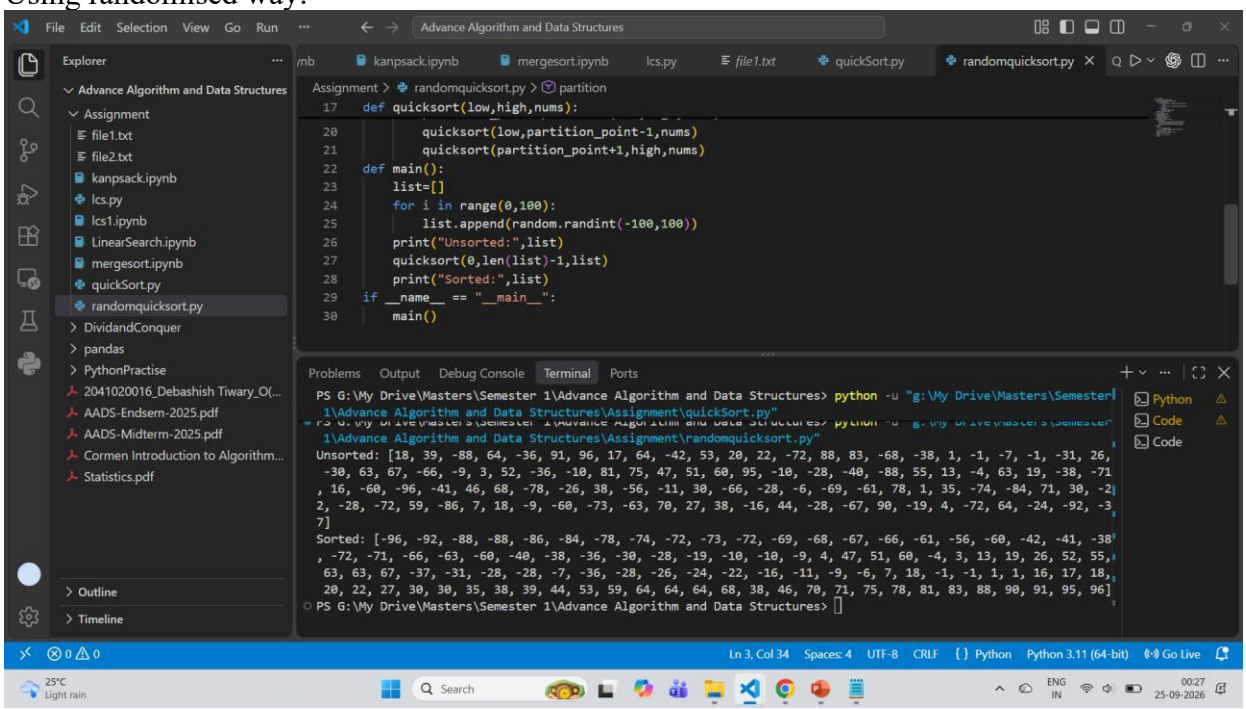
Time Complexity: Quick Sort takes O(N^2) time in worst cases(for already for almost sorted data) whereas the avg case is O(NlogN) N- size of data

Space Complexity: It takes an extra space of O(1) but auxiliary space due to recursion call.

OUTPUT:



Using randomised way:



EXPERIMENT-06

AIM: To implement the Fractional Knapsack using the Greedy Approach technique in Python.

Theory:

The **Fractional Knapsack problem** is a classic optimization problem in computer science where the goal is to fill a container (a "knapsack") with a combination of items to maximize the total value without exceeding the knapsack's weight capacity.

*The idea is to always pick items greedily based on their **value-to-weight ratio**. Take the item with the highest ratio first, then the next highest, and so on, until the knapsack is full. If any item doesn't fully fit, then take its fractional part according to the remaining capacity.*

Steps to solve the problem:

1. Calculate the ratio (value/weight) for each item.

2. Sort all the items in decreasing order of the ratio.

3. Iterate through items:
   if the current item fully fits, add its full value and decrease capacity otherwise, take the fractional part that fits and add proportional value.

4. Stop once the capacity becomes zero

Code:

```
data=[['A',6000,650],['B',5750,700],['C',4000,500],['D',3000,300]
] for i in data:
 i.append(i[2]/i[1]*100)
data=sorted(data,key= lambda x:x[3],reverse=True)
fund = int(input("Entre Fund Amount"))
def knapsack(data,fund):
 totalreturn=0
 for i in data:
   if(fund>0 and fund//i[1]!=0):
         totalreturn+=i[2]
         fund=fund-i[1]
   elif(fund>0):
         totalreturn+=(fund*i[3]/100)
         fund=0
```

return totalreturn

Complexity Analysis:

Time Complexity: It generally takes O(NlogN) time mainly dominated by the sorting approach that we take.

Space Complexity: It takes an extra space of O(1) or O(N) based on situation or desired outcomes.

OUTPUT:

